MACHINE LEARNING

External Practical Exam — Complete Revision Guide

9 Practicals • Python Code • Key Concepts • Viva Questions • Common Viva

Quick preparation tip: Understand the aim, dataset, main code lines, output and one application for each practical.

Prepared for last-minute practical revision

How to use this guide

For every practical, prepare these five things: (1) Aim, (2) definition/theory, (3) code flow, (4) output and evaluation, and (5) applications. The code examples below are demonstration examples; use your college notebook's exact dataset and code if it differs.

Quick map of the 9 practicals

No.	Practical	Key points
1	Data Preprocessing	Missing values, duplicates, encoding, scaling
2	Linear Regression	Continuous prediction, MSE, R^2
3	Gradient Descent	Cost function, gradient, learning rate
4	Logistic Regression	Classification, sigmoid, accuracy
5	Decision Tree	Root/leaf, entropy, Gini, overfitting
6	KNN	K value, distance, feature scaling
7	K-Means	Clusters, centroid, unsupervised learning
8	Association Rule Learning	Apriori, support, confidence, lift
9	Elbow Method	WCSS, selecting K

Common Python libraries

pandas — data manipulation; **NumPy** — numerical calculations; **scikit-learn** — ML models, preprocessing and metrics; **Matplotlib** — plots; **mlxtend** — association rules and Apriori.

Practical 1 — Data Preprocessing

Aim: To study and implement data preprocessing techniques using a student dataset.

Theory: Data preprocessing cleans and transforms raw data into a suitable format before machine learning. Common tasks include handling missing values, removing duplicates, encoding categorical variables and scaling features.

Python code (demonstration)

```
import pandas as pd
import numpy as np

df = pd.read_csv("Student.csv")
print(df.head())
print(df.info())
print(df.isnull().sum())

df = df.drop_duplicates()
df = df.fillna(df.mean(numeric_only=True))

print(df.head())
print(df.shape)
```

Code/output notes: The fillna(mean) line fills missing numeric cells with each numeric column's mean. If categorical columns have missing values, handle them separately (for example, with the mode).

Viva questions

What is data preprocessing?

Answer: Cleaning and transforming raw data before model training.

Why use drop_duplicates()?

Answer: To remove duplicate rows.

What does isnull().sum() do?

Answer: Counts missing values in each column.

What is the difference between dropna() and fillna()?

Answer: dropna removes rows/columns with missing data according to its settings; fillna replaces missing values.

What is feature scaling?

Answer: Transforming numeric features to comparable scales.

What does df.shape return?

Answer: A tuple containing the number of rows and columns.

What is encoding?

Answer: Converting categorical values into numerical representations.

Practical 2 — Linear Regression

Aim: To implement and evaluate a Linear Regression model.

Theory: Linear Regression is a supervised learning algorithm that predicts a continuous numerical target by modelling a linear relationship between input features and output.

Python code (demonstration)

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

df = pd.DataFrame({
    "Hours": [1, 2, 3, 4, 5],
    "Marks": [20, 30, 40, 50, 60]
})
X = df[["Hours"]]
y = df["Marks"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Predicted:", y_pred)
print("MSE:", mean_squared_error(y_test, y_pred))
print("R2:", r2_score(y_test, y_pred))
```

Code/output notes: X contains input features and y contains the target. Lower MSE is generally better; R^2 describes how much target variation is explained by the model on the evaluated data.

Viva questions

What is Linear Regression used for?

Answer: Predicting continuous numerical values, such as prices.

What is the equation?

Answer: For one feature: y = mx + c.

What is fit()?

Answer: It trains the model.

What is predict()?

Answer: It predicts outputs for input data.

What is MSE?

Answer: The mean of squared differences between actual and predicted values.

What is R^2?

Answer: A measure of how well the model explains variation in the target.

Why split train and test data?

Answer: To evaluate performance on data not used for training.

Practical 3 — Gradient Descent

Aim: To implement Gradient Descent for cost-function minimization.

Theory: Gradient Descent is an optimization algorithm that repeatedly updates model parameters in the direction that reduces the cost function.

Python code (demonstration)

```
import numpy as np

X = np.array([1, 2, 3, 4, 5], dtype=float)
y = np.array([2, 4, 6, 8, 10], dtype=float)

m, c = 0.0, 0.0
learning_rate = 0.01
iterations = 1000
n = len(X)

for i in range(iterations):
    y_pred = m * X + c
    dm = (-2/n) * np.sum(X * (y - y_pred))
    dc = (-2/n) * np.sum(y - y_pred)
    m = m - learning_rate * dm
    c = c - learning_rate * dc

print("Slope:", m)
print("Intercept:", c)
print("Prediction for X=6:", m*6 + c)
```

Code/output notes: Learning rate controls the update size; too high can cause divergence, while too low makes learning slow. Iterations are repeated update steps.

Viva questions

What is Gradient Descent?

Answer: An optimization algorithm for minimizing a cost function.

What is a cost function?

Answer: A function that quantifies prediction error.

What is learning rate?

Answer: The step size used to update parameters.

What if learning rate is too high?

Answer: The algorithm may overshoot or fail to converge.

What are iterations?

Answer: Repeated steps of parameter updates.

Is Gradient Descent itself a supervised algorithm?

Answer: No; it is an optimization method used to train many kinds of models.

Practical 4 — Logistic Regression

Aim: To implement and evaluate a Logistic Regression model.

Theory: Logistic Regression is a supervised algorithm mainly used for classification. The sigmoid function maps a real-valued score to a value between 0 and 1.

Python code (demonstration)

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

data = load_breast_cancer()
X, y = data.data, data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
model = make_pipeline(
    StandardScaler(),
    LogisticRegression(max_iter=5000)
)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
```

Code/output notes: The pipeline scales features using training data before fitting Logistic Regression. Accuracy alone may be insufficient for imbalanced datasets.

Viva questions

What is Logistic Regression used for?

Answer: Mainly classification.

What is the sigmoid function?

Answer: It maps a real-valued input to a value between 0 and 1.

What is binary classification?

Answer: Classification into two classes.

What is a confusion matrix?

Answer: A table summarizing correct and incorrect predictions by class.

Difference between Linear and Logistic Regression?

Answer: Linear Regression predicts numerical values; Logistic Regression is mainly for classification.

What is accuracy?

Answer: Correct predictions divided by total predictions.

Practical 5 — Decision Tree Classification

Aim: To implement and evaluate a Decision Tree classification algorithm.

Theory: A Decision Tree makes predictions using a tree-like structure of feature-based decisions. It can be used for classification and regression.

Python code (demonstration)

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

data = load_iris()
X, y = data.data, data.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Code/output notes: Root is the top node; internal nodes split data; leaf nodes give predictions. Entropy and Gini impurity are split-quality measures.

Viva questions

What is a Decision Tree?

Answer: A tree-based supervised algorithm for classification or regression.

What is a root node?

Answer: The topmost node.

What is a leaf node?

Answer: A terminal node that gives a prediction.

What is entropy?

Answer: A measure of impurity or uncertainty.

What is Gini impurity?

Answer: A measure of class impurity at a node.

What is overfitting?

Answer: Fitting training data too closely and generalizing poorly.

Can a Decision Tree do regression?

Answer: Yes, with a Decision Tree Regressor.

Practical 6 — K-Nearest Neighbors (KNN)

Aim: To implement and evaluate the KNN classification algorithm.

Theory: KNN predicts a sample's class using the labels of its K nearest training samples. Distance-based methods are sensitive to feature scales.

Python code (demonstration)

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

data = load_iris()
X, y = data.data, data.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = make_pipeline(
    StandardScaler(),
    KNeighborsClassifier(n_neighbors=3)
)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Code/output notes: K is the number of neighbors. Very small K can be noisy; very large K can underfit. StandardScaler is fitted only on training data through the pipeline.

Viva questions

What does K represent?

Answer: The number of nearest neighbors used.

Is KNN supervised?

Answer: Yes.

What is Euclidean distance?

Answer: Straight-line distance between points.

Why scale features?

Answer: To prevent features with larger ranges dominating distance.

What is a lazy learner?

Answer: A method that stores examples and does much of its work during prediction.

What if K is too small or too large?

Answer: Too small can overfit/noise; too large can underfit.

Practical 7 — K-Means Clustering

Aim: To implement and evaluate the K-Means clustering algorithm.

Theory: K-Means is an unsupervised algorithm that partitions data into K clusters by assigning points to nearby centroids and updating the centroids.

Python code (demonstration)

```
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans

X, _ = make_blobs(n_samples=100, centers=3, random_state=42)
model = KMeans(n_clusters=3, random_state=42, n_init=10)
labels = model.fit_predict(X)

plt.scatter(X[:, 0], X[:, 1], c=labels)
plt.scatter(
    model.cluster_centers_[:, 0],
    model.cluster_centers_[:, 1],
    marker="X", s=200
)
plt.title("K-Means Clustering")
plt.show()
```

Code/output notes: K is the chosen number of clusters; centroid is the mean position of cluster points. Scaling may be needed for features with different ranges.

Viva questions

Is K-Means supervised?

Answer: No, it is unsupervised.

What is clustering?

Answer: Grouping similar data points.

What is a centroid?

Answer: The mean position of points in a cluster.

What does n_clusters=3 mean?

Answer: Create three clusters.

What does fit_predict() do?

Answer: Fits the clustering model and returns cluster labels.

Classification vs clustering?

Answer: Classification uses labelled classes; clustering discovers groups.

Practical 8 — Association Rule Learning

Aim: To implement Association Rule Learning using frequent itemsets and association rules.

Theory: Association Rule Learning discovers co-occurrence relationships among items. Apriori finds frequent itemsets; support, confidence and lift help evaluate rules.

Python code (demonstration)

```
# Install once in Jupyter if needed:
# %pip install mlxtend

import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

transactions = [
    ["Bread", "Milk"],
    ["Bread", "Butter"],
    ["Bread", "Milk", "Butter"],
    ["Milk", "Butter"],
    ["Bread", "Milk"]
]

te = TransactionEncoder()
encoded = te.fit(transactions).transform(transactions)
df = pd.DataFrame(encoded, columns=te.columns_)

itemsets = apriori(df, min_support=0.4, use_colnames=True)
print(itemsets)

rules = association_rules(
    itemsets, metric="confidence", min_threshold=0.5
)
print(rules[["antecedents", "consequents",
             "support", "confidence", "lift"]])
```

Code/output notes: Support is the fraction of transactions containing an itemset. Confidence estimates P(B|A). Lift compares confidence with the support of B; lift > 1 indicates positive association relative to independence.

Viva questions

What is Association Rule Learning?

Answer: Discovering relationships among items in transactions.

What is Apriori?

Answer: An algorithm for finding frequent itemsets and generating rules.

What is support?

Answer: Fraction of transactions containing an itemset.

What is confidence?

Answer: Fraction of A-containing transactions that also contain B for A→B.

What is lift?

Answer: Confidence divided by support of the consequent.

What is market basket analysis?

Answer: Finding products that are frequently bought together.

What does min_support=0.4 mean?

Answer: Keep itemsets appearing in at least 40% of transactions.

Practical 9 — Elbow Method

Aim: To determine a suitable number of clusters using the Elbow Method.

Theory: The Elbow Method plots K-Means WCSS/inertia for several K values. The elbow is a point where adding clusters begins to give diminishing improvement.

Python code (demonstration)

```
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans

X, _ = make_blobs(n_samples=200, centers=4, random_state=42)
wcss = []

for k in range(1, 11):
    model = KMeans(n_clusters=k, random_state=42, n_init=10)
    model.fit(X)
    wcss.append(model.inertia_)

plt.plot(range(1, 11), wcss, marker="o")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("WCSS")
plt.title("Elbow Method")
plt.show()
```

Code/output notes: WCSS is the sum of squared distances from each point to its assigned centroid. It generally decreases as K increases; the elbow is a heuristic, not always unambiguous.

Viva questions

Why use the Elbow Method?

Answer: To estimate a suitable K for K-Means.

What is WCSS?

Answer: Within-Cluster Sum of Squares.

Why does WCSS decrease as K increases?

Answer: More centroids usually reduce distances to assigned centroids.

What is the elbow point?

Answer: A point after which extra clusters give diminishing WCSS improvement.

Is the elbow always clear?

Answer: No, some datasets have no obvious elbow.

How are K-Means and Elbow Method related?

Answer: K-Means forms clusters; the Elbow Method helps choose K.

Common Machine Learning Viva

What is Machine Learning?

Answer: A branch of AI that learns patterns from data to make predictions or decisions.

Name three types of Machine Learning.

Answer: Supervised learning, unsupervised learning and reinforcement learning.

What is supervised learning?

Answer: Learning from labelled input-output examples.

What is unsupervised learning?

Answer: Finding patterns or groups in data without target labels.

What are features and target?

Answer: Features are inputs; the target is the value/class being predicted.

What is a dataset?

Answer: A collection of related data used for analysis or model development.

What is training?

Answer: Learning model parameters or patterns from training data.

Why test a model?

Answer: To estimate performance on unseen data.

What is overfitting?

Answer: A model fits training data too closely and generalizes poorly.

What is underfitting?

Answer: A model is too simple to capture important patterns.

What is regression?

Answer: Predicting continuous numerical values.

What is classification?

Answer: Predicting a discrete class/category.

What is clustering?

Answer: Grouping similar data points.

What is scikit-learn?

Answer: A Python library for machine learning and preprocessing.

What is Pandas?

Answer: A Python library for data manipulation and analysis.

What is NumPy?

Answer: A Python library for numerical computing.

What is Matplotlib?

Answer: A Python library for plotting and visualization.

What does train_test_split do?

Answer: Splits data into training and testing subsets.

What does random_state do?

Answer: Makes randomized splitting or initialization reproducible.

What is model.fit(X_train, y_train)?

Answer: Fits/trains the model on the training features and targets.

What does model.predict(X_test) do?

Answer: Generates predictions for test inputs.

What is accuracy?

Answer: Correct predictions divided by the total number of predictions.

What is the difference between a parameter and a hyperparameter?

Answer: Parameters are learned from data; hyperparameters are set before/during training.

Why is data quality important?

Answer: Poor or incorrect data can produce unreliable models.

Last-minute examiner checklist

• Memorize the aim and one-sentence definition for all 9 practicals.

• Be able to explain X (features) and y (target).

• Explain fit(), predict(), train_test_split() and the metric used.

• Know whether each algorithm is supervised or unsupervised.

• Know the expected output: printed values, metric, cluster labels or graph.

• For Practical 1, know how missing values and duplicates are handled.

• For Practical 8, memorize support, confidence and lift formulas.

• For Practical 9, explain WCSS and the elbow point.

• Use your college notebook's exact code and dataset if they differ from these examples.

• Run each code once before the exam if time permits.

Honest exam tip: No guide can guarantee full marks, but understanding the code line-by-line and practicing the actual notebook programs gives you the strongest preparation.